

Caching*

June 1, 2020

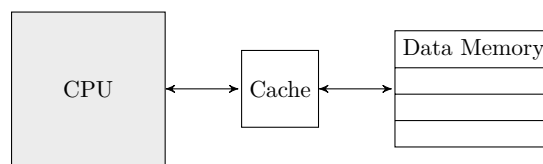
1 Caching

Previously we discussed how data gets from storage or memory to the CPU, and identified that this issue of getting data to the CPU is a bottleneck for how fast a computer can work. In this section, we present one approach to “widening” the bottleneck by using *cache*s.

Caching is a concept that comes into play frequently in computer science. We’re particularly concerned with caching within a CPU for now, but similar concepts arise in every scenario where you might utilize caching.

To set the stage for a conversation about caching: data is always stored in *words*, or chunks of bits (for example, 8 bits, or 32 bits— usually a power of 2). Caching is all about moving these words from one place in memory or storage to another place.

A *cache memory* is a relatively small, specialized memory device placed between the CPU and memory, as shown in figure 1. The cache memory holds *copies* of words from memory that seem likely to be accessed by the processor. The cache is faster than main memory; thus, if frequently accessed locations are found in the cache, the average memory access time will be small. The strategy followed by the cache is to hold data at or near locations recently accessed by the processor. The *locality of reference* exploited by this strategy is the propensity of memory accesses, over short periods of time, to cluster in small regions of memory.



Data in the cache are redundant, in the sense that each is used to provide a more accessible copy of information also stored in slower main memory. Thus the total addressable memory size, as seen by the programmer, is not increased by the presence of a cache. Rather, the cache provides, in a program-transparent way, an improvement in the average memory access time.

Because the cache is much smaller than main memory, only some of the main-memory locations can be cached at any one time. Consequently, in the general case each location in the cache, or *cache line* conceptually has two parts: a *tag* field and a *contents* field. The tag field helps to identify the physical address from the virtual address of that cache line. The contents field contains the data that lives in that physical address. When a read or write operation is requested by the

*Based on *Computer Systems: A Programmer’s Perspective* and *Computation Structures*

CPU, the accessed location's address (which is a physical address) is compared with the tag field of selected cache lines. If a match is found (that is, the tag indicates that the cache line maps to the desired address), the contents field of the cache line containing the matching tag is read or written. There is no need to access main memory (which takes more time than accessing the data in the cache). This is known as a **cache hit**.

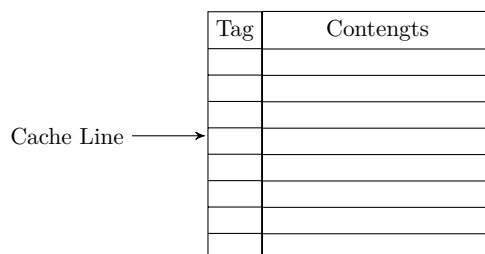


Figure 1: This cache has 8 cache lines.

If the address does not match any tag, a **cache miss** occurs. If the cache misses on a **read**, the datum is fetched from main memory, returned to the processor, and also stored in one of the cache lines (by setting the selected cache line's tag to the address accessed and the selected cache line's contents to the value received from main memory). Since the cache is of limited size, this updating must replace some value already stored there. The **replacement strategy** for choosing which of these values to replace affects the effectiveness of the cache.

For a memory **write**, both the cache and the main memory must eventually be updated. Many cache-memory systems update main memory immediately. This technique is called **memory write-through**. The processor doesn't wait for the write to main memory to complete before proceeding; the write to main memory happens concurrently with subsequent cache reads.

Another commonly used technique is to defer the main memory write until the modified cache is replaced. This approach is more complicated, but usually reduces the number of write operations to main memory, which is important when bandwidth to main memory is a critical resource. This is called a **write-back**.

2 Different Ways to Organize a Cache

The general idea of caching is easy to understand, but as always, it's the actual design and implementation where the details get complex. In this section, we'll explore the various approaches to organizing a cache. Generally, this reduces to: how to map the cache's **virtual address** to the main memory's **physical address**; how big the cache is; when to put data in the cache; and finally, how the cache works when *writing* to memory rather than reading from it.

As with all engineering problems, there are tradeoffs among the approaches. The relevant variables here are *cost* (\$\$), *time (data access) improvement*, and *complexity*.

There are three main ways to organize a cache, which form a continuum. The organization strategies primarily refer to the relationship between virtual addresses and physical addresses. In order to be efficient, it needs to be easy (and fast!) to convert between the physical and virtual addresses. The first form of organization is **direct mapped**, where each physical address is allowed to be put in a single location in the cache (there's a specific mapping between many physical addresses and one virtual address). On the other end of the continuum is a **fully associative**

cache: Each position in the cache can map to any physical address in main memory. As you can imagine, in this case, more structure is required to map between physical and virtual addresses.

In the middle is the *set-associative* cache, which is a combination of the two.

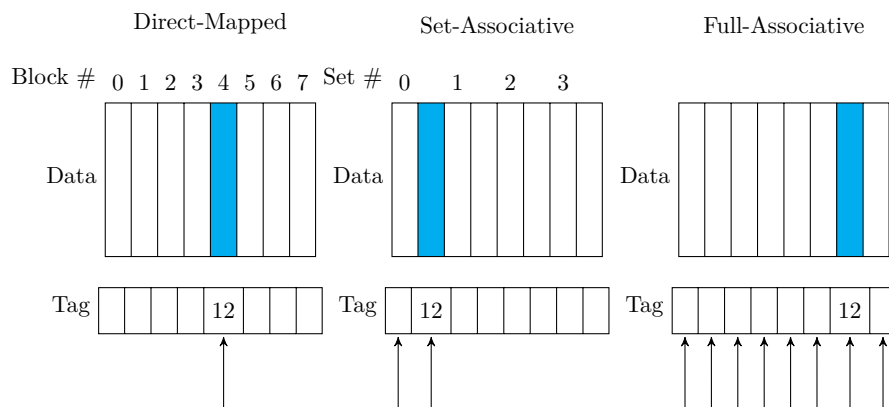


Figure 2: Examples of all three caches. The arrows indicate which cache lines are inspected when accessing a data address. With Direct-Mapped, there's a single cache line to look at; in the set, it's 2 (the size of the set). In the Fully-Associative, all cache lines need to be checked.

2.1 Fully Associative Cache

When a cache miss occurs, the cache line selected to hold the data read from main memory can in general be any cache line, if a replacement strategy such as LRU is in use. Since the contents of a given main-memory location might reside in any cache line, it is necessary to examine the tag field of each line in the cache to determine whether a given memory access constitutes a cache hit. Although the inspection of cache tag fields could conceivably be performed sequentially, performance considerations generally require that all tag fields be simultaneously compared with an incoming address, delaying access on a hit by *one* comparison time rather than by $\Theta(N)$ comparison times. This parallelism involves $\Theta(N)$ hardware cost (that is, build this in hardware rather than use software), since an independent comparator is required for each cache line. It's not shown in the figure (and is outside of the scope of this class), but the requested address is compared sent (via hardware) to EACH line in the cache, and compared to the tag. We assume that the cache contents are maintained so as to avoid duplicate tag fields; thus the incoming address will match at most one of the tags. If there is such a match (that is, if there is a cache hit), then (in hardware) a HIT signal fires, and the content portion of the corresponding cache line is written to the output for use by the CPU.

Here's an example: If location 12 is resident in the cache, some cache line will contain 12 in its tag field and $\langle 12 \rangle$ (the contents of memory location 12) in its contents field. A read of address 12 by the processor (or CPU) will lead to a match at the comparator associated with that cache line, driving HIT active and asserting $\langle 12 \rangle$ as the cache output data. The CPU (or associated logic) will interpret the cache output data as the result of the read operation, and the main memory will not be accessed. If, however, no cache line contains (12), then no tag will match and HIT will not be signaled. In this case, some more circuitry (hardware; not shown in the diagram) will fetch $\langle 12 \rangle$ from main memory, replacing some selected cache line with the tag 12 and contents $\langle 12 \rangle$.

Time	Address accessed	Cache Block	Hit or Miss	Cache			
				0	1	2	3
1	0		Miss	M[0]			
2	8		Miss	M[0]	M[8]		
3	0		Hit	M[0]	M[8]		
4	6		Miss	M[0]	M[8]	M[6]	
5	8		Hit	M[0]	M[8]	M[6]	

$M[x]$ = data in the memory location x

Figure 3: An example of a series of memory accesses using a fully associative cache. When an address is accessed, we check all of the 4 cache lines to determine if they contain data from the desired address. If not, we add the desired data to the cache. Here, there are 3 misses.

This cache may be viewed as a memory system in which locations are named by a portion of their contents (the tag) rather than by addresses fixed to each location. Such systems are termed *content-addressable* or *associative memories* and constitute an abstraction whose usefulness can be used in a variety of situations, not just memory caches.

In general, an associative memory of size N embodies, in a simple abstraction, an N -way parallel search. Truly associative memories can be and have been made; however, their cost remains quite high because of the replication of comparison logic (lots of complicated circuits!). As a result, fully associative caches of the type sketched here are rarely constructed in practice. Practical cache approaches almost invariably achieve lower cost by reducing the amount of parallelism.

2.2 Direct-Mapped Cache (Nonassociative)

While a fully associative cache of reasonable size is prohibitively expensive due to the complex hardware circuits, it can be made *very* much cheaper by sacrificing parallelism so as to allow the use of ordinary RAMs as the storage medium. An extreme example of this approach is sketched in figure 16.9, which illustrates a cache that examines a single location of an ordinary RAM to determine whether a processor access causes a cache hit. The cache is therefore not associative; it is sometimes referred to as a direct-mapped cache, since main-memory addresses are mapped into cache-memory addresses in a straightforward way.

Note that the direct-mapped cache uses *low-order* (the right-most) bits of the incoming memory address to dictate the address within the cache to examine for a hit. Thus memory location A can only reside in the cache line whose address is the low k bits of A , a substantial constraint compared with the fully associative alternative. It is this constraint—the discipline that the address used within the cache is dictated by the main-memory address to be cached—that allows the use of addressable RAM rather than associative memory. (Observe that, since the low-order bits of a main-memory address A are implicit in the identity of the direct-mapped cache line that holds $\langle A \rangle$, it is unnecessary to include these bits in the stored tag or in the comparison.)

The constraint implies, however, that memory locations that share the same k low-order address bits also share the same cache line. Among other limitations, such a cache cannot support an LRU replacement strategy; indeed, the cache line to be replaced on a miss is dictated entirely by the new address to be cached. This inadequacy of the direct-mapped cache exhibits itself when two or more simultaneous “hot spots” in the memory access pattern of an executing program—say, the

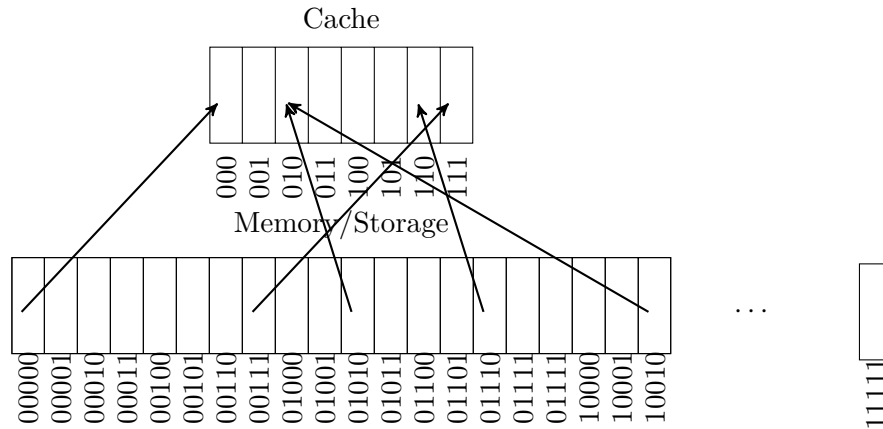


Figure 4: A direct-mapped cache. What happens when we populate the cache with data from 010101 or 10010? How do we know which address the data came from?

instruction addresses of a tight loop and the addresses of data manipulated by that loop— compete for the same cache lines. The result is frequent replacement of cached data about to be accessed, with a dramatic drop in the hit ratio. The chances of such a collision between hot spots decreases as the total size of the cache grows. Since a non-associative cache uses ordinary (albeit fast) RAMs, it can be made quite large for the cost of even a small fully associative cache. While the collision probability never becomes negligible, a large direct-mapped cache can perform remarkably well; it is often the cache of choice on a performance/cost basis. Its major drawback is the unpredictability of its performance: A seemingly trivial program change can redistribute addresses so as to cause (or cure) a serious collision problem, making a conspicuous and unnerving difference in the running time of the program.

Address accessed	Cache Block	Hit or Miss	0	1	2	3
0	$(0 \bmod 4) = 0$	Miss	M[0]			
8	$(8 \bmod 4) = 0$	Miss	M[8]			
0	$(0 \bmod 4) = 0$	Miss	M[0]			
6	$(6 \bmod 4) = 2$	Miss	M[0]		M[2]	
8	$(8 \bmod 4) = 0$	Miss	M[8]		M[2]	

$M[x]$ = data in the memory location x

Figure 5: Overall, we have 5 misses.

2.3 Set-Associative Cache

A commonly used intermediate point on the cache associativity spectrum is the N-way set-associative cache sketched in figure 16.10. Such a cache may be viewed as N nonassociative caches operating in parallel. The incoming address is bussed to each of the N comparators, while its low-order bits dictate single locations in each of N RAMs to be accessed. Thus every main-memory location

The set-associative cache combines most of the cost advantages of the non-associative cache with some associativity to avoid collisions in typical programs. Since reference patterns of typical programs contain only a few independent hot spots at any time, a four-way set-associative cache of sufficient size will typically eliminate the danger of serious collisions, since four independent ranges of consecutive addresses can be cached simultaneously.

Figure 6: Different set-associative caches

	Tag Data	
0		
1		
2		
3		
4		
5		
6		
7		

Two-way set associative

	Tag	Data	Tag	Data
0				
1				
2				
3				

	TagData	TagData	TagData	TagData
0				
1				

[illegible]

So far, we've talked about the big picture of caching, and some various ways to design a cache. But, how to decide which approach to take? As with all engineering problems, there are pros and cons, tradeoffs between approaches. There are situations where one cache design provides more improvement than another. How do we evaluate a cache design?

Let the fast memory's access time be t_{af} , its size be M_f , and its cost per bit be c_f . Similarly, let the access time of the slow memory be t_{as} , its size be M_s , and its cost per bit be c_s . Then the effective size of the memory will be approximately M_s , and its cost will be $c_f \cdot M_f + c_s \cdot M_s$. If a

Address accessed	Cache Block	Hit or Miss	0	0	1	1
0	$(0 \bmod 2) = 0$	Miss	M[0]			
8	$(8 \bmod 2) = 0$	Miss	M[0]	M[8]		
0	$(0 \bmod 2) = 0$	Hit	M[0]	M[8]		
6	$(6 \bmod 2) = 0$	Miss	M[0]	M[6]		
8	$(8 \bmod 2) = 0$	Miss	M[6]	M[8]		

$M[x]$ = data in the memory location x

Figure 7: Overall, we have 4 misses.

is the fraction of all memory accesses that address only the fast memory, then the average access time of the memory as a whole will be

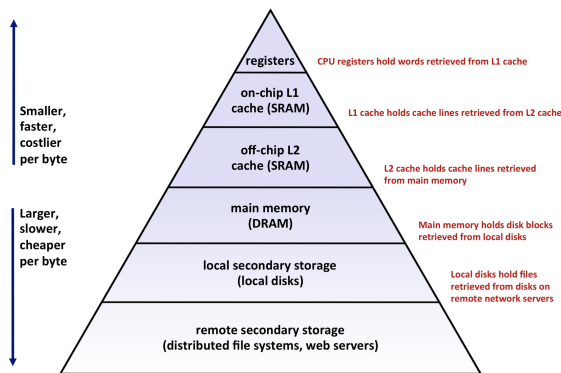
$$t_a = \alpha - t_{af} + (1 - \alpha) - t_{as}. \quad (1)$$

If α is close to 1, then t_a will be close to t_{af} . In multilevel-memory jargon, α is called the *hit ratio*, or hit rate. The complementary *miss ratio* $1 - \alpha$ reflects the fraction of memory transactions that require access to the slower memory. (For this equation we assume that both the fast and slow memory accesses can be started at the same time; if the slow memory access does not begin until it is found that the desired datum is not in the fast memory, then t_{as} must include the time for the probe of fast memory as well as the access time of the slower memory in order for the equation to be valid.)

3 The Memory Hierarchy

- Big Idea: The memory hierarchy creates a large pool of storage that costs as much as the cheap storage near the bottom, but that serves data to programs at the rate of the fast storage near the top.
- Fundamental idea of a memory hierarchy:
 - For each k , the faster, smaller device at level k serves as a cache for the larger, slower device at level $k + 1$.
- Why do memory hierarchies work?
 - Because of locality, programs tend to access the data at level k more often than they access the data at level $k + 1$.
 - Thus, the storage at level $k + 1$ can be slower, and thus larger and cheaper per bit.
- Some fundamental and enduring properties of hardware and software:
- Fast storage technologies cost more per byte, have less capacity, and require more power (heat!).
- Well-written programs tend to exhibit good locality.

- They suggest an approach for organizing memory and storage systems known as a memory hierarchy.



4 Caching and Code

All of this caching stuff is quite well and nice, but if a cache is supposed to be more or less invisible to the programmer, why do we care about this?

Well, even though we programmers can't explicitly control what happens with the cache (all of that is done in hardware and the OS), we *can* write code that either allows the cache to do its job, or that makes the cache useless. Too much of this is outside the scope of this course, but I want you to know that it does matter.

This section will cover some examples of how caching and programming can work together or not.

5 Cache Latency

The reason we even worry about caching is due to latency. See the table to get a sense of some latency numbers you may find interesting.

6 Caching all kinds of things

I noted at the beginning that caching happens in lots of places in Computer Science. Table 6 gives a sense of this.

At the top of the table are the CPU registers, where the processor is working. The TLB, L1, L2 and L3 caches are all (usually) part of the CPU, and they sit between the processor and main memory. In addition to the L* caches, there's also Virtual Memory, which is designed similarly to the caches we've been talking about, but are bigger and slower than the L* caches.

Now we start to get further from the CPU: when accessing files, we'll start caching parts of the file; same with the disk (because disks are slow!), the network (so slow!), and so on.

7 Closing Remarks

Takeaways:

Latency Comparison Numbers (~2012) ¹

L1 cache reference	0.5 ns			
Branch mispredict	5 ns			
L2 cache reference	7 ns			14x L1 cache
Mutex lock/unlock	25 ns			
Main memory reference	100 ns			20x L2 cache, 200x L1 cache
Compress 1K bytes with Zippy	3,000 ns	3 us		
Send 1K bytes over 1 Gbps network	10,000 ns	10 us		
Read 4K randomly from SSD*	150,000 ns	150 us		1GB/sec SSD
Read 1 MB sequentially from memory	250,000 ns	250 us		
Round trip within same datacenter	500,000 ns	500 us		
Read 1 MB sequentially from SSD*	1,000,000 ns	1,000 us	1 ms	1GB/sec SSD, 4X memory
Disk seek	10,000,000 ns	10,000 us	10 ms	20x datacenter roundtrip
Read 1 MB sequentially from disk	20,000,000 ns	20,000 us	20 ms	80x memory, 20X SSD
Send packet CA->Netherlands->CA	150,000,000 ns	150,000 us	150 ms	

1 ns = 10⁻⁹ seconds

1 us = 10⁻⁶ seconds = 1,000 ns

1 ms = 10⁻³ seconds = 1,000 us = 1,000,000 ns

Type	What Cached
CPU Registers	4-byte or 8-byte words
TLB	Address translations
L1 Cache	64-byte blocks
L2 Cache	64-byte blocks
L3 cache	64-byte blocks
Virtual Memory	4-kb pages
Buffer Cache	Parts of files
Disk Cache	Disk sectors
Network Cache	Parts of files
Browser Cache	Web pages
Web Cache	Web pages

- We put a cache between the CPU and memory to help improve the CPU performance.
- A cache is a mini-memory.
- A cache has virtual addresses that map to physical addresses.
- When a piece of data in the main memory/data storage is accessed, it gets put into the cache.
- Because of locality, caching in general helps make things faster.
- Caching details:
 - the cache is smaller than the main memory.
 - How to know where a piece of data in the cache came from?
 - How to choose where in the cache a piece of data goes?

- When the cache is full, how to decide which piece of data gets evicted to allow the new piece of data?
 -
- We've been talking about data, but the same techniques are used for instructions.
- We're talking about a *cache* between the CPU and main memory, but similar ideas apply in many contexts (hashtables!)

8 Problems

We'll work through a problem to help us understand how caches work.

To start, we'll consider a number of alternative cache designs; for each, we'll look at a code snippet and explore how the cache impacts performance.

1. We're going to use a test program to walk through different caches. For this problem, we assume that *instructions* are loaded separately from data; we are ONLY focused on data accesses here (simplicity).

Consider the following C function:

```
1 int N = 4;
2 int A[N];
3 int B[N];
4
5 register int a, b;
6
7 int ctr = 1000;
8
9 while (ctr > 0) {
10     int x = N;
11     while (x > 0) {
12         a = A[x]; // Load the value A[x] into register a
13         b = B[x]; // Load the value B[x] into register b
14         x--;
15     }
16     ctr--;
17 }
```

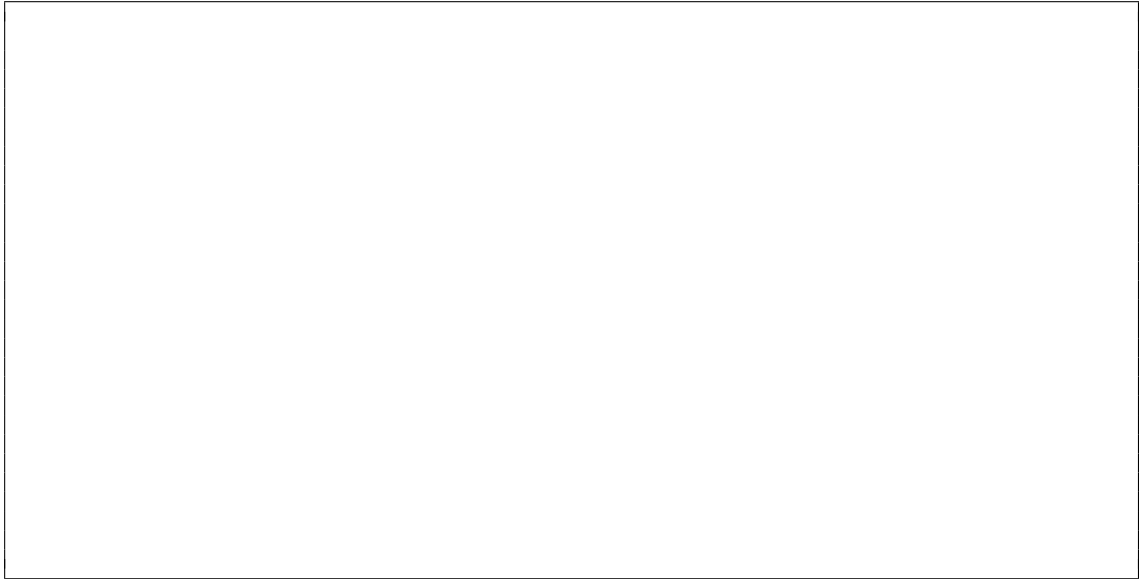
Assume that the array *A* begins at memory location 0x2000 and array *B* begins at memory location 0x3000.

Assume that an `int` is 4 bytes.

- All caches have a 1-word (4-byte) block size.
- Each cache is used for *all* memory accesses, including instructions.
- Cache *W*: 64-line direct mapped cache
- Cache *X*: two-way set-associative cache; random replacement, total 64 lines.
- Cache *Y*: four-way set-associative cache; random replacement, total 64 lines.
- Cache *Z*: fully associative cache, total size 64 lines, LRU replacement.

Note: all caches have the same capacity, and can store 64 words of data.

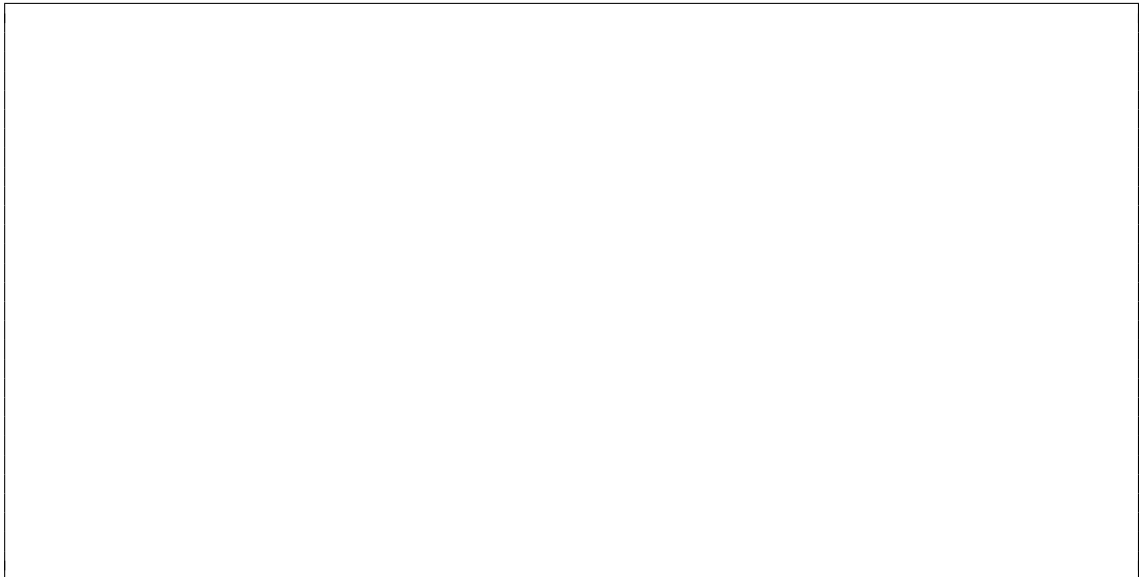
- (a) Which configuration results in the highest hit ratio?



(b) Assume the following access times:

-
- A cache hit has an access time of 50 ns.
- A cache miss requires 250ns.
- A write-through strategy is used, with 180 ns for each data write.
- The cache is initially empty before the program begins.

How many nanoseconds of the above program's execution will be spent waiting for memory, for each of the 4 caches?



Some alternative configurations to consider:

- What happens if the array *A* begins at memory location 0x00 and array *B* begins at memory location 0x00? (that is, they are the same array)

- What happens if $N = 64$?

2. This problem concerns the cache shown below.

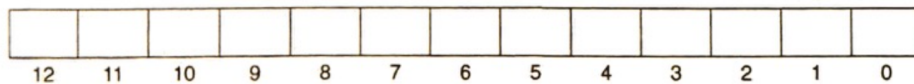
Assume the following:

- The memory is byte-addressable
- Memory accesses are to 1-byte words (not to 4-byte words)
- Addresses are 12 bits wide
- The cache is 2-way set associative ($E = 2$), with a 4-byte block size ($B = 4$) and four sets ($S = 4$).

Set index	Tag	Valid	Byte 0	Byte 1	Byte 2	Byte 3
0	00	1	40	41	42	43
	83	1	FE	97	CC	D0
1	00	1	44	45	46	47
	83	0	—	—	—	—
2	00	1	48	49	4A	4B
	40	0	—	—	—	—
3	FF	1	9A	C0	03	FF
	00	0	—	—	—	—

(a) The following diagram shows the format of an address (1 bit per box). Indicate (by labeling the diagram) the fields that would be used to determine the following:

- CO: The cache block offset
- CI: The cache set index
- CT: The cache tag



(oops, ignore bit 12!)

(b) For each of the memory accesses, indicate if it will be a cache hit or miss when carried out in sequence as listed. Also give the value of a read if it can be inferred from the information in the cache.

Operation	Address	Hit?	Read value (or unknown)
Read	0x834	_____	_____
Write	0x386	_____	_____
Read	0xFFD	_____	_____

3. This problem concerns the cache shown below.

Assume the following:

- The memory is byte-addressable
- Memory accesses are to 1-byte words (not to 4-byte words)
- Addresses are 13 bits wide
- The cache is 2-way set associative ($E = 2$), with a 4-byte block size ($B = 4$) and eight sets ($S = 8$).

2-way set associative cache												
Set index	Line 0						Line 1					
	Tag	Valid	Byte 0	Byte 1	Byte 2	Byte 3	Tag	Valid	Byte 0	Byte 1	Byte 2	Byte 3
0	09	1	86	30	3F	10	00	0	—	—	—	—
1	45	1	60	4F	E0	23	38	1	00	BC	0B	37
2	EB	0	—	—	—	—	0B	0	—	—	—	—
3	06	0	—	—	—	—	32	1	12	08	7B	AD
4	C7	1	06	78	07	C5	05	1	40	67	C2	3B
5	71	1	0B	DE	18	4B	6E	0	—	—	—	—
6	91	1	A0	B7	26	2D	F0	0	—	—	—	—
7	46	0	—	—	—	—	DE	1	12	C0	88	37

(a) List all of the hex memory addresses that will hit in set 1.

(b) List all of the hex memory addresses that will hit in set 6.

(c) List all of the hex memory addresses that will hit in set 2.

(d) List all of the hex memory addresses that will hit in set 4.

(e) What data is in address 0111 0101?

(f) What address holds the piece of data “88”?

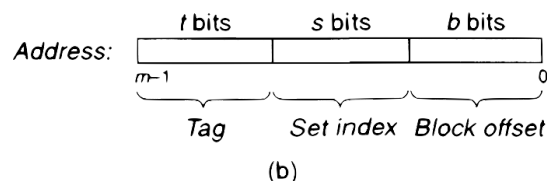
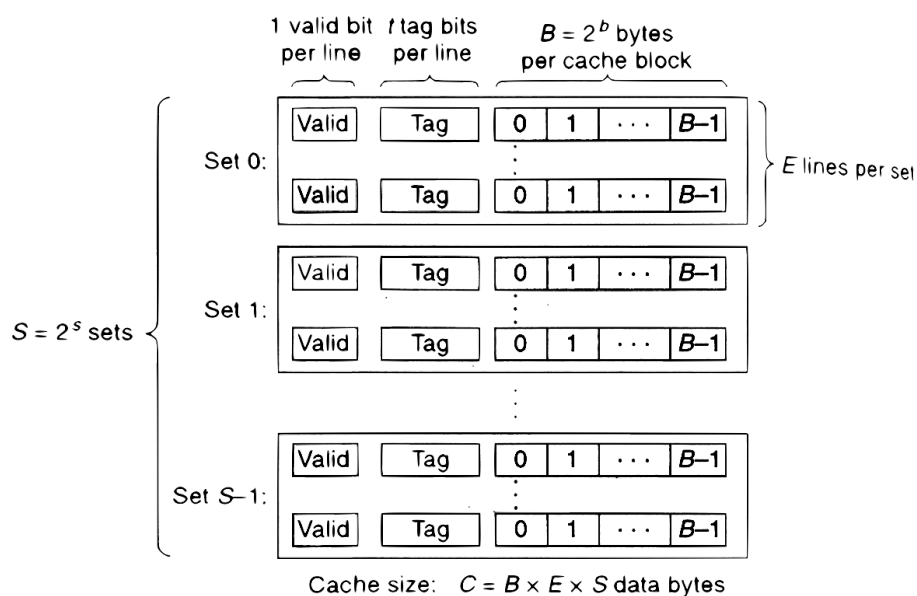
--

4. The following table gives the parameters for a number of different caches.

For each cache, fill in the missing fields in the table.

For reference, the fundamental parameters of a cache (from the reading, page 616-617):

Parameter	Description
Fundamental parameters	
$S = 2^s$	Number of Sets
E	Associativity
$B = 2^b$	Block size (bytes)
$m = \log_2(M)$	Number of physical (main memory) address bits
Derived quantities	
$M = 2^m$	Maximum number of unique memory addresses
$s = \log_2(S)$	Number of <i>set index bits</i>
$b = \log_2(B)$	Number of <i>block offset bits</i>
$t = m - (s + b)$	Number of <i>tag bits</i>
$C = B \times E \times S$	Cache size (bytes), not including overhead such as valid and tag bits



	m	C	B	E	S	t	s	b
1	32	1024	4	4	_____	_____	_____	_____
2	32	1024	4	256	_____	_____	_____	_____
3	32	1024	8	1	_____	_____	_____	_____
4	32	1024	8	128	_____	_____	_____	_____
5	32	1024	32	1	_____	_____	_____	_____
6	32	1024	32	4	_____	_____	_____	_____
7	32	_____	8	1	_____	21	8	3
8	32	2048	_____	_____	128	23	7	2
9	32	1024	2	8	64	_____	_____	1
10	32	1024	_____	2	16	23	4	_____

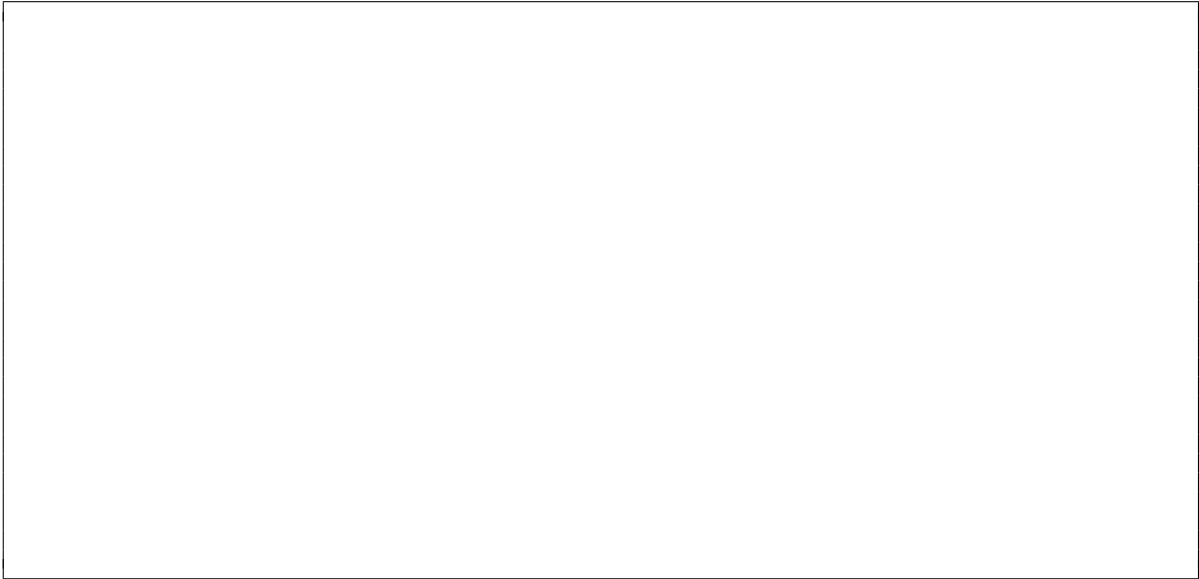
$$\text{Capacity} = \frac{\# \text{ bytes}}{\text{sector}} \times \frac{\text{avg } \# \text{ sectors}}{\text{track}} \times \frac{\# \text{ tracks}}{\text{surface}} \times \frac{\# \text{ surfaces}}{\text{platter}} \times \frac{\# \text{ platters}}{\text{disk}}$$

$$x = x$$

5. Estimate the average time (in ms) to access a sector on the following disk:

Parameter	value
Rotational Rate	15,000 RPM
$T_{avgseek}$	4ms
Avg num sectors/track	800

6. Suppose you are asked to design a rotating disk where the number of bits per track is constant. You know that the number of bits per track is determined by the circumference of the innermost track, which you can assume is also the circumference of the hole. Thus, if you make the hole in the center of the disk larger. The number of bits per track increases, but the total number of tracks decreases. If you let r denote the radius of the platter, and x r the radius of the hole. what value of x maximizes the capacity of the disk?

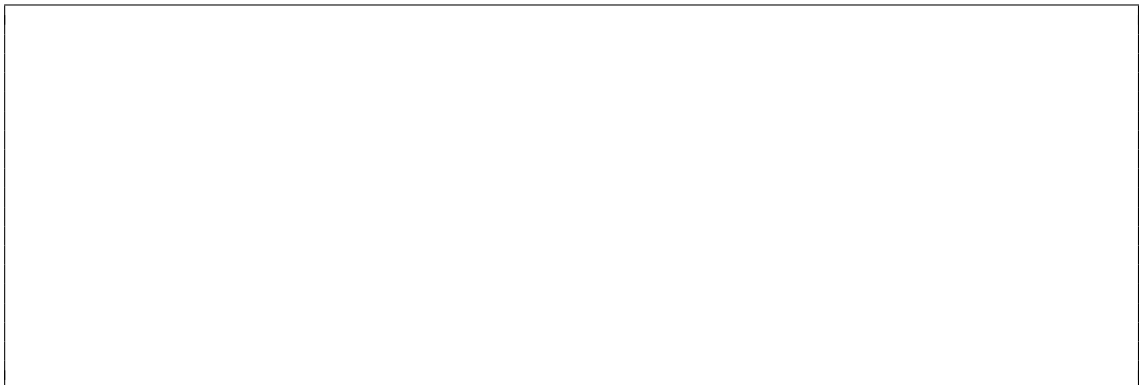


7. Suppose that a 2 MB file consisting of 512-byte logical drive with the following characteristics:

Parameter	value
Rotational Rate	15,000 RPM
$T_{avgseek}$	4ms
Avg num sectors/track	800
Surfaces	8
Sector Size	512 bytes

For each case below, suppose that a program reads the logical blocks of the file sequentially, one after the other, and that the time to position the head over the first block is $T_{avgseek} + T_{avgrotation}$

- (a) Best case: Estimate the optimal time (in ms) required to read the all possible mappings of logical blocks to disk sectors. file over



- (b) Random case: Estimate the time (in ms) required to read the file if are mapped randomly to disk sectors.

